# Declarative Capability Language (DCL)

## A Position Paper on Declarative Architectural Intent

**Version:** 1.0 **Status:** Position Paper **Date:** July 2026

# Abstract

Software architecture is typically described through a combination of natural language, diagrams, architectural decision records (ADRs) and implementation. While these artefacts communicate intent effectively between people, much of their meaning remains implicit and difficult to analyse automatically.

The Declarative Capability Language (DCL) explores an alternative approach. Rather than describing how software is implemented, DCL provides a declarative language for expressing what a system is responsible for, the behaviour it guarantees, the outcomes it produces and the operational constraints under which those behaviours must exist.

The language is centred on business capabilities rather than implementation constructs such as services, classes or infrastructure. It deliberately separates architectural intent from implementation decisions, allowing intent to be compiled, analysed and reasoned about independently of technology choices.

DCL is an implemented language with a compiler, semantic analysis, intermediate representation (IR), Visual Studio Code extension, MCP server and supporting documentation. This paper describes the motivation behind the language, its core design principles and the questions it raises for software architecture, programming language design and AI-assisted engineering.

This paper is intended as an invitation for informed critique rather than a claim of novelty.

# 1. Introduction

Software architecture is fundamentally concerned with intent.

Architects describe what a system should achieve, the responsibilities it should own, the constraints under which it should operate and the qualities it must exhibit over time.

Today that intent is communicated through documents, diagrams, decision records, conversations and eventually source code.

Each artefact captures part of the picture.

None captures the complete semantic model.

The result is that architectural intent gradually becomes distributed across multiple representations, making it difficult to validate, analyse or evolve consistently.

DCL explores a different question:

> Can architectural intent itself become a compilable artefact?

Rather than replacing implementation, DCL attempts to provide an explicit semantic representation that sits between architectural thinking and software implementation.

---

# 2. Motivation

Most architecture documentation communicates intent through prose.

This works well for human discussion but presents challenges for automated reasoning.

Questions such as:

- Which business capability owns this behaviour?
- Which outcomes are guaranteed?
- What policies govern execution?
- Which effects are permitted?
- What invariants must always hold?
- What causes a particular outcome?

are often answered only indirectly.

The implementation eventually answers many of these questions, but only after architectural decisions have already been translated into technology-specific structures.

DCL investigates whether these questions can instead be answered directly from an implementation-independent language.

---

# 3. Design Philosophy

Several principles guided the development of DCL.

## Implementation Independence

The language intentionally excludes implementation concerns including programming languages, frameworks, deployment models, cloud platforms and infrastructure topology.

Different implementations may realise the same capability while sharing an identical architectural contract.

---

## Capabilities as the Architectural Unit

DCL treats the business capability as the primary architectural abstraction.

Capabilities remain relatively stable as technologies evolve.

Implementation structures are expected to change.

This distinction allows architectural behaviour to remain stable even as software evolves.

---

## Semantics Before Syntax

The language is designed around meaning rather than textual representation.

The compiler constructs an explicit semantic model that can be analysed independently of the surface syntax.

---

## Explicit Behaviour

Behaviour is expressed through:

- intent
- outcomes
- actors
- rules
- invariants
- effects
- lifecycle
- policies

rather than through imperative implementation.

---

# 4. The Language

DCL models architectural behaviour using a small set of core concepts.

Capabilities describe responsibilities.

Contexts provide bounded scope.

Intent expresses why behaviour exists.

Outcomes define successful completion.

Actors identify responsibility.

Rules and invariants constrain behaviour.

Effects describe externally observable consequences.

Policies define operational expectations.

Lifecycle describes behavioural progression over time.

Together these constructs form an implementation-independent semantic description of architectural behaviour.

---

# 5. Compiler and Semantic Analysis

DCL is more than a textual notation.

Source programs are compiled into an intermediate semantic representation from which additional analysis can be performed.

Current tooling includes:

- parsing
- semantic validation
- intermediate representation (IR)
- capability summaries
- dependency analysis
- cause-and-effect analysis
- architectural visualisations
- compiler diagnostics

The compiler therefore operates on architectural meaning rather than solely on syntax.

---

# 6. Example

This paper intentionally includes only a simplified example.

A complete worked example is provided separately.

The purpose here is simply to illustrate that DCL expresses architectural behaviour independently of implementation technology.

```
language dcl 1.0

actor Customer is human
actor SupportAgent is agent
actor CRMSystem is system

shape CustomerQuestion {
  customerId: Uuid required
  question: Text required
}

shape AnswerDraft {
  customerId: Uuid required
  answer: Text required
  confidence: Number required
}

shape EscalationRequest {
  customerId: Uuid required
```

```
  reason: Text required
}

shape KnowledgeSearchResult {
  summary: Text required
  source: Text required
}

effect SearchKnowledgeBase is tool
effect CheckCustomerAccount is tool
effect CreateSupportTicket is invocation
effect NotifyHumanSupport is notification

event CustomerQuestionAnswered is AnswerDraft
event SupportQuestionEscalated is EscalationRequest

policy MinimumAnswerConfidence {
  confidence {
    threshold 0.8
  }
}

policy SafeToolRetry {
  reliability {
    retry {
      attempts 2
      backoff exponential
    }
    idempotency required
  }
}

policy AuditSupportAnswer {
  governance {
    audit required
    evidence required
  }
}

capability AnswerCustomerQuestion {
  intent CustomerQuestion from SupportAgent

  outcomes {
    AnswerPrepared is AnswerDraft
    Escalated
    InsufficientConfidence
    ToolUnavailable
  }

  effects {
    SearchKnowledgeBase
    CheckCustomerAccount after SearchKnowledgeBase
  }
```

```
  events {
    emits CustomerQuestionAnswered
  }

  policies {
    MinimumAnswerConfidence governs outcome AnswerPrepared
    SafeToolRetry governs effect SearchKnowledgeBase
    AuditSupportAnswer governs outcome AnswerPrepared
  }

  when {
    SearchKnowledgeBase failed then ToolUnavailable
    CheckCustomerAccount failed then Escalated
    policy MinimumAnswerConfidence fails then InsufficientConfidence
    otherwise then AnswerPrepared
  }

  lifecycle {
    begin Received
    step Investigating
    step Answered
    step Escalated
    end Resolved

    move Received to Investigating on outcome AnswerPrepared
    move Investigating to Answered on event CustomerQuestionAnswered
    move Investigating to Escalated on outcome Escalated
    move Answered to Resolved on event CustomerQuestionAnswered
    move Escalated to Resolved on event SupportQuestionEscalated
  }
}

capability EscalateSupportQuestion {
  intent EscalationRequest from SupportAgent

  outcomes {
    EscalationCreated
    EscalationRejected
  }

  effects {
    CreateSupportTicket
    NotifyHumanSupport after CreateSupportTicket
  }

  events {
    emits SupportQuestionEscalated
  }

  policies {
    SafeToolRetry governs effect CreateSupportTicket
  }

  when {
```

```
      CreateSupportTicket failed then EscalationRejected
      otherwise then EscalationCreated
    }
  }
```

---

# 7. Current Tooling

The current implementation includes:

- DCL Compiler
- Intermediate Representation (IR)
- Analysis Passes
- Visual Studio Code Extension
- MCP Server
- Documentation
- Website
- Examples

The language is currently released as Version 1.0.

---

# 8. An Unexpected Observation

DCL was not originally created with artificial intelligence in mind.

Its initial purpose was to provide an explicit representation of architectural intent that could be validated independently of implementation.

However, during development it became apparent that large language models appeared to reason over DCL significantly more effectively than equivalent architectural prose.

This observation has motivated further exploration into whether structured semantic representations may improve AI-assisted software engineering.

No formal evaluation has yet been undertaken, and this remains an area for future investigation.

---

# 9. Relationship to Existing Work

DCL should not be interpreted as replacing:

- Architecture Description Languages
- UML
- Architectural Decision Records
- Requirements specifications
- Source code

Instead it explores whether there is value in representing architectural intent as an explicit semantic contract that can coexist with these artefacts.

Determining where DCL aligns with existing research, where it differs and where it overlaps remains an important area for further study.

---

# 10. Open Questions

This work raises several questions that would benefit from external review.

- Does capability-centred modelling provide a useful architectural abstraction?
- Which existing languages or modelling approaches most closely resemble DCL?
- Which semantic concepts require stronger formalisation?
- Can implementation-independent architectural intent improve automated reasoning?
- Can architectural intent become a reliable basis for conformance and assurance?

These questions remain intentionally open.

---

# 11. Invitation for Critical Review

DCL has been developed as an open-source language and supporting toolchain.

The intention of this paper is not to claim that DCL represents a new class of programming language or architectural model.

Rather, it is an invitation to researchers and practitioners to critically evaluate the language, identify relevant prior work, challenge its assumptions and help determine where it may contribute to existing research and engineering practice.

Constructive criticism is welcomed.

---

# Resources

- Website: *https://russelleast.github.io/Capability-Language/*
- GitHub Repository: *https://github.com/russelleast/Capability-Language*
- Language Reference: *https://russelleast.github.io/Capability-Language/docs/*
- Examples: *https://russelleast.github.io/Capability-Language/examples/*

---

# About the Author

Russell East is a Principal Software Architect with over thirty years of software engineering experience spanning enterprise systems, distributed architectures and cloud-native platforms.

DCL emerged from practical architectural work rather than academic research, motivated by the challenge of expressing architectural intent in a precise, implementation-independent form. The language has since evolved into a standalone compiler, tooling ecosystem and open-source project.

Russell is seeking informed critique from researchers and practitioners to help evaluate DCL within the broader landscape of programming languages, software architecture and AI-assisted engineering.

---

# Appendix A — Design Timeline

## The Evolution of DCL

DCL did not begin as a programming language research project.

It emerged from practical software architecture, where a recurring challenge became apparent: architectural intent was distributed across documents, diagrams, Architectural Decision Records (ADRs), source code and conversations. While each artefact captured part of the picture, none represented the architecture as a coherent, machine-understandable model.

The language evolved incrementally as successive architectural problems were identified and addressed.

| Stage | Evolution | Motivation |
|---|---|---|
| **Architectural Discovery** | Recognition that architectural intent was fragmented across multiple artefacts. | Existing documentation communicated architecture effectively to people but was difficult to analyse, validate or evolve consistently. |
| **Capabilities as the Primary Abstraction** | Shift from describing systems as services or components to describing them as business capabilities. | Capabilities proved to be a more stable architectural boundary as implementations evolved. |
| **Behaviour Over Structure** | Introduction of intent, outcomes, actors, rules and effects. | The focus moved from describing software structure to expressing behavioural responsibility. |
| **Semantic Contracts** | Addition of invariants, policies and lifecycle semantics. | Architectural expectations became explicit and compiler-verifiable rather than remaining implicit within documentation. |
| **Compiler Development** | Construction of a compiler and intermediate representation (IR). | DCL became an analysable language rather than a descriptive notation. |
| **Semantic Analysis** | Introduction of compiler diagnostics and analysis passes. | Architectural models could be checked for ambiguity, consistency, reachability and completeness. |
| **Tooling Ecosystem** | Development of the VS Code extension, documentation, | DCL became practical for day-to-day architectural modelling and AI-assisted |

| Stage | Evolution | Motivation |
|---|---|---|
| | visualisations and MCP server. | development. |
| **AI-Assisted Engineering** | Observation that large language models reasoned effectively over DCL models. | This was an unexpected outcome rather than an original design objective, leading to new research questions around semantic representations for AI-assisted engineering. |

## Evolution of the Design Goals

The objectives of DCL evolved alongside the language itself.

**Initial objective**

> Provide a more precise way of expressing architectural intent.

↓

**Second objective**

> Enable architectural intent to be validated through compilation and semantic analysis.

↓

**Third objective**

> Provide a stable, implementation-independent representation that can evolve independently of software technology.

↓

**Current exploration**

> Investigate whether explicit semantic representations improve automated reasoning, architectural conformance and AI-assisted engineering.

## Design Principles That Emerged

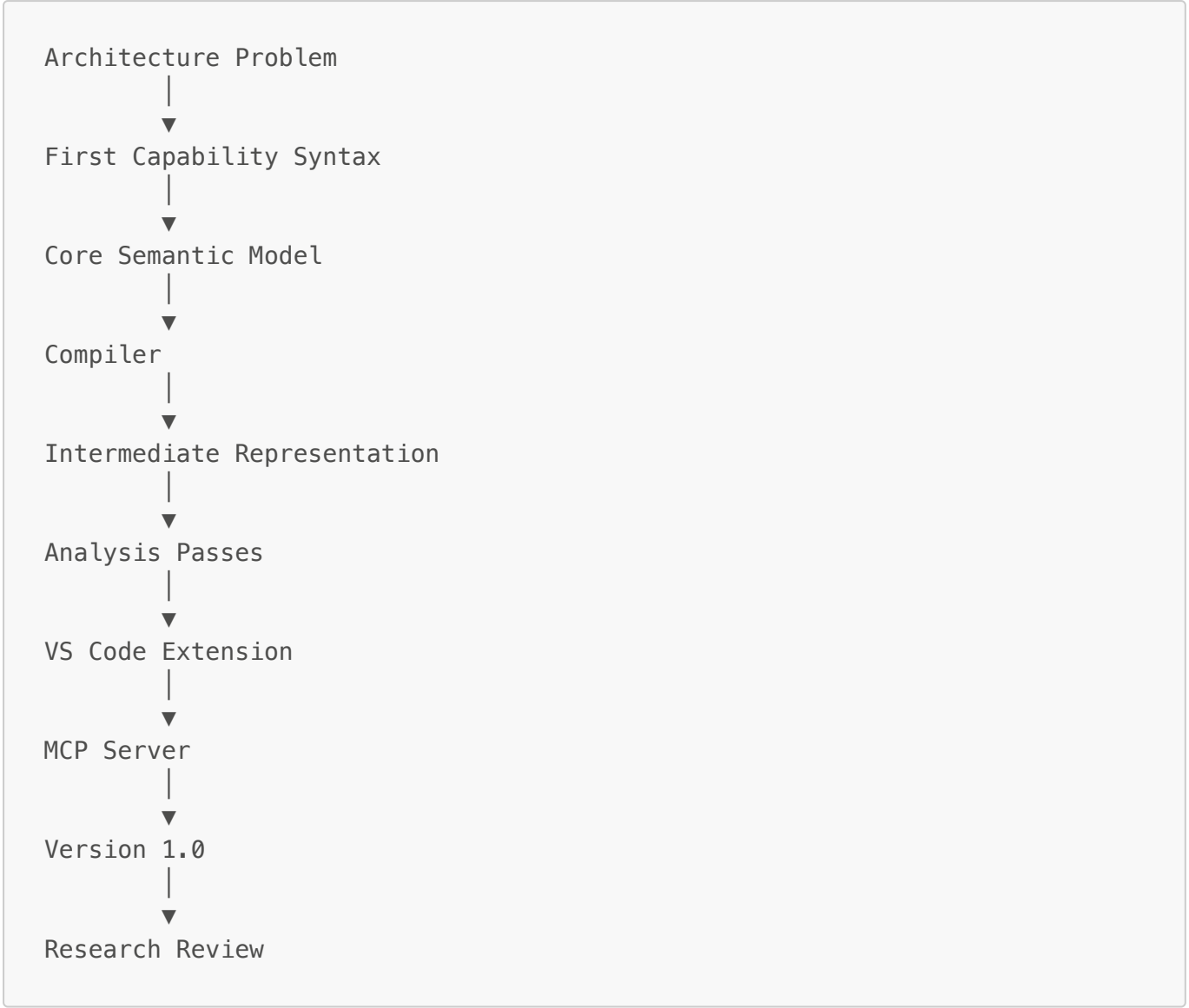
Several of DCL's defining principles were not established at the outset but emerged through successive iterations of the language.

- Implementation independence
- Capability-centred modelling
- Behaviour before implementation
- Explicit semantic contracts
- Compiler-validated architectural intent
- Human-readable and machine-analysable representations

These principles now underpin both the language design and its supporting tooling.

## Language Evolution

The development of DCL progressed through recognisable stages of language engineering.




The progression illustrates that DCL evolved from an architectural idea into a complete language ecosystem with compiler tooling, editor integration and supporting infrastructure before seeking external review.

## Reflection

One of the most significant observations during the development of DCL was that its usefulness extended beyond its original purpose.

Although conceived as a language for expressing architectural intent, DCL has shown promise as a semantic representation that can be consumed by development tools and AI systems. This was not an explicit design objective but an emergent property of making architectural behaviour explicit, structured and compiler-validated.

Whether DCL represents a meaningful contribution to programming languages, software architecture or AI-assisted engineering remains an open question. The purpose of this paper is to invite informed critique that will help answer that question.